

QP-3.2 - Linguagens de Programação: Questões pré-classe para a aula 2 de "Nomes, Vinculações, Verificações de Tipo e Escopo"

Total de pontos 28/28 ?

E-mail *

lucascruvinel@discente.ufcat.edu.br

0 de 0 pontos

Qual é o seu nome? (para registro das respostas) *

LUCAS ELIAS DE ANDRADE CRUVINEL

2 de 2 pontos

✓ Uma variável cuja amarração de armazenagem ocorre na área de pilha * 1/1

- ☐ ocupa a memória para armazenar os valores retornados por uma função (ou subprograma).
- ☐ ocupa a memória para armazenar os valores passados como parâmetro para uma função (ou subprograma).
- ☒ passa a ocupar espaço em memória na chamada do subprograma (função, procedimento ou método) na qual foi definida até que a chamada o subprograma termine. ✓
- ☐ passa a ocupar espaço em memória na chamada do subprograma (função, procedimento ou método) na qual foi definida até que ocorra a próxima chamada de subprograma.

✓ Quando ocorre uma chamada recursiva a uma mesma função, a amarração de armazenagem das variáveis nela declaradas *

1/1

- ☐ ocorre a cada chamada sucessiva realizada desde que a variável seja declarada como global, fazendo com que seja alocado o armazenamento N vezes para a mesma variável quando ocorrer N chamadas recursivas (encadeadas) da função.
- ☒ ocorre a cada chamada sucessiva realizada, fazendo com que seja alocado o armazenamento N vezes para a mesma variável quando ocorrer N chamadas recursivas (encadeadas) da função. ✓
- ☐ ocorre uma única vez quando a especificação da função é lida pelo compilador ou interpretador.
- ☐ ocorre uma vez pelo conjunto de chamadas sucessiva, fazendo com que seja alocado o armazenamento um única vezes a variável quando ocorrer N chamadas recursivas (encadeadas) da função.

9 de 9 pontos

Para a função mdc abaixo, uma invocação mdc(5,3) faz com que a pilha armazene os seguintes valores para tmp para cada uma chamadas, nesta ordem:

```
int mdc(int x, int y) {  
    int tmp = x % y;  
    if (tmp == 0)  
        return y;  
    else  
        return mdc(y, tmp);  
}
```



✓ Quais são os valores armazenados na pilha para tmp? *

1/1

- ☐ 2, 1
- ☐ 1, 0
- ☒ 2, 1, 0
- ☐ 3
- ☐ 2
- ☐ 5, 3, 1



✓ Ao comparar a amarração de memória de variáveis de uma função recursiva à sua função equivalente não recursiva (iterativa, implementada apenas com laços de for/while), constatamos que a função recursiva ocupa mais memória. Para uma chamada recursiva que causa N chamadas subsequentes e a equivalente iterativa que causa M iterações do(s) laço(s), é possível dizer que (a) a alocação das variáveis locais da função recursiva e (b) as das variáveis usadas especificamente nas sentenças da solução iterativa, ocorrerá respectivamente: *

1/1

- ☒ N vezes e 1 vez.
- ☐ N vezes para cada uma das variáveis e M vezes para todas as variáveis.
- ☐ N vezes e M/N vezes.
- ☐ N vezes e M vezes.
- ☐ N*M vezes e M vezes.



✓ Em uma variável de heap, o endereço de memória ao qual ela será amarrado é definido * 1/1

- ☐ durante a ligação (linkedição) do código.
- ☐ durante a compilação do código.
- ☐ no momento da especificação da linguagem.
- ☒ em tempo de execução do código. ✓
- ☐ no momento da codificação do programa.

✓ Uma variável alocada explicitamente na heap é uma variável para qual o programador * 1/1

- ☐ acessa diretamente a área de memória para a qual a variável foi amarrada.
- ☐ indica no código o endereço desejado para alocação na heap.
- ☒ indica no código o momento de alocação na heap e a variável representa o endereço de memória alocado. ✓
- ☐ indica no código o valor que será armazenado na variável, deixando para o compilador especificar o endereço onde ele será armazenado.

✓ O tipo de uma variável alocada explicitamente na heap é amarrado na * 1/1

- ☐ execução, pois é necessário saber quais são as porções da heap livres para realizar a alocação.
- ☐ execução, pois é necessário saber quanto espaço precisa ser alocado para a variável.
- ☐ compilação, pois é necessário saber qual a variável para a qual será associado o espaço na heap.
- ☒ compilação, pois é necessário saber quanto espaço precisa ser alocado para a variável. ✓

✓ Para uma variável alocada implicitamente na heap, o programador não controla * 1/1

- ☐ o momento da alocação da heap. A desalocação depende se a linguagem oferece desalocação explícita ou implícita via coletor de lixo.
- ☒ o momento da alocação e desalocação da heap. ✓
- ☐ o momento da alocação da heap, podendo controlar ou não o momento da desalocação.
- ☐ o momento da desalocação, podendo controlar ou não o momento da alocação.

✓ Em uma linguagem de programação, o escopo de uma variável especifica * 1/1

- ☐ a faixa de sentenças em que uma variável está ocupando memória.
- ☒ o(s) bloco(s) de códigos em que uma variável é visível. ✓
- ☐ todas as demais afirmações estão corretas.
- ☐ o trecho de código em que uma variável foi declarada.

✓ O que é uma variável local a um bloco de sentenças B? * 1/1

- ☒ É uma variável visível, portanto no escopo de B, e que foi declarada no bloco de sentenças B. ✓
- ☐ É uma variável declarada no programa principal no qual o bloco B está inserido.
- ☐ É uma variável acessível dentro do escopo de B.
- ☐ É uma variável manipulada dentro do escopo de B.



✓ O que é uma variável não-local a um bloco de sentenças B? *

1/1

- ☒ É uma variável visível, portanto no escopo de B, e que foi declarada fora do bloco de sentenças B. ✓
- ☐ É uma variável não-acessível dentro do escopo de B.
- ☐ É uma variável manipulada fora do escopo de B.
- ☐ É uma variável declarada no programa principal no qual o bloco B está inserido.

6 de 6 pontos

Considere o código abaixo em C:

```
int a;

int f() {
    int b = 0;
    int c = 1;
    return b + c;
}

void main() {
    int d;
    int e;
    f();
}
```

✓ Quais são as variáveis locais e não-locais da função main()? *

1/1

- ☒ Locais: d, e; Não-locais: a ✓
- ☐ Locais: d, e, a; Não-locais: nenhuma
- ☐ Locais: d, e; Não-locais: a, b, c
- ☐ Locais: d, e, a; Não-locais: b, c



✓ Para o mesmo trecho de código anterior, quais são as variáveis locais e não locais da função f()? *

1/1

- ☐ Locais: b, c; Não-locais: d, e, a
- ☐ Locais: b, c, a; Não-locais: d, e
- ☐ Locais: b, c, d, e; Não-locais: a
- ☒ Locais: b, c; Não-locais: a



✓ Em uma linguagem de programação que permite escopo estático, as variáveis visíveis em uma função são *

1/1

- ☐ As variáveis globais, também chamadas de variáveis estáticas de um programa.
- ☒ As variáveis definidas na própria função (locais), acrescidas das variáveis do escopo de todos os blocos/subprogramas na qual aquela função se insere, inclusive o escopo global (que define variáveis globais).
- ☐ Unicamente as variáveis definidas na própria função (locais).
- ☐ Definidas pelo escopo antes do programa executar.



✓ Em uma linguagem de programação que permite escopo dinâmico, o conjunto de variáveis visíveis em uma função é definida

1/1

- ☒ pela ordem de chamada dos subprogramas (exemplo: funções) na qual a função é chamada, acrescida das variáveis locais e globais.
- ☐ pelo compilador durante a execução do programa.
- ☐ unicamente pelas variáveis com amarração dinâmica de nomes.
- ☐ pelas variáveis com amarração dinâmica de nomes e tipos.
- ☐ pelas variáveis com amarração dinâmica de nomes, tipos e armazenagem.



✓ Há duas variáveis com mesmo nome declaradas como variável global e como variável local a uma função. O que ocorrerá com acesso dessas duas variáveis? *

1/1

- ☒ A variável global será ocultada pela variável local da função, deixando portanto de ser acessada ou visível. ✓
- ☐ A variável local será ocultada pela variável global da função, deixando portanto de ser acessada ou visível.
- ☐ O nome da variável local e global referenciarão a mesma variável em memória.
- ☐ Tipicamente ocorrerá um erro de compilação ou interpretação do código.

✓ Em uma linguagem dinâmica, a amarração de tipo e armazenagem das variáveis ocorre em tempo de execução. Por que nessas linguagens é necessário oferecer uma palavra reservada (exemplo: `global` do Python) para acessar um variável global? *

1/1

- ☒ Porque o interpretador precisa ser capaz de diferenciar essa variável global de uma variável local dinamicamente declarada de uma variável global. ✓
- ☐ Para permitir o acesso à variável local com o mesmo nome.
- ☐ Para permitir o escopo dinâmico. Do contrário, a linguagem só permitiria escopo estático.

1 de 1 pontos

Considere o seguinte código C com mais de um bloco interno a outro, definindo portanto seu próprio escopo.

```
int f1(int p) {
    int a;
    int b;
    a = p+1;
    b = p*p;
    if (p > 1) {
        int a = p-1;
        while (1) {
            if (a >= b) {
                break; // sai do while
            }
            a = a + 1;
        }
    }
}
```




```
}  
  return a;  
}
```

✓ Qual é o valor retornado por f1(2) *

1/1

☐ 4☐ 5☐ 2☒ 3

2 de 2 pontos

Considere o seguinte trecho de código de uma linguagem hipotética com escopo dinâmico.

```
int a = 1  
int b = 2  
  
int f1() {  
  int a = 4  
  return f2()  
}  
  
int f2() {  
  int b = 8  
  return f3()  
}  
  
int f3() {  
  return a + b  
}
```



✓ Quais serão os valores retornados para as chamadas f1(), f2() e f3()? * 1/1

- ☐ 6, 9, 3
- ☐ 3, 3, 3
- ☐ nenhum das demais alternativas está correta.
- ☒ 12, 9, 3 ✓
- ☐ 12, 12, 3
- ☐ 12, 3, 3

✓ Para o mesmo código da questão anterior, quais seriam os valores retornados para as chamadas f1(), f2() e f3() se o escopo fosse estático? 1/1

- *
- ☐ 6, 9, 3
- ☒ 3, 3, 3 ✓
- ☐ 12, 3, 3
- ☐ nenhum das demais alternativas está correta.
- ☐ 12, 9, 3
- ☐ 12, 12, 3

3 de 3 pontos

Considere o seguinte código de uma linguagem hipotética com escopo dinâmico

```
void f1(int a) {  
  if (a > 0) {  
    f2()  
  }  
  else {  
    f3()  
  }  
}
```



```
void f2() {  
    int a = 0  
    int b = 1  
    f4()  
}
```

```
void f3() {  
    int b = 3  
    int c = 5  
    f4()  
}
```

```
void f4() {  
    print(a)  
    print(b)  
    print(c)  
}
```

✓ Quais seriam os valores impressos (via função print()) para a chamada f1(0) e f1(1)? * 1/1

- ☐ f1(0): 0, 3, 5 - f1(1): 1, 1, 5
- ☐ f1(0): 0, 3, 5 - f1(1): ERRO EM TEMPO DE EXECUÇÃO, ERRO EM TEMPO DE EXECUÇÃO, ERRO EM TEMPO DE EXECUÇÃO
- ☐ f1(0): 0, 1, 5 - f1(1): ERRO EM TEMPO DE EXECUÇÃO, ERRO EM TEMPO DE EXECUÇÃO, ERRO EM TEMPO DE EXECUÇÃO
- ☐ f1(0): 0, 3, 5 - f1(1): 1, 1, ERRO EM TEMPO DE EXECUÇÃO
- ☒ f1(0): 0, 3, 5 - f1(1): 0, 1, ERRO EM TEMPO DE EXECUÇÃO ✓
- ☐ f1(0): 0, 3, 5 - f1(1): 0, 1, 5
- ☐ f1(0): 0, 1, 5 - f1(1): 1, 3, 5
- ☐ f1(0): ERRO EM TEMPO DE EXECUÇÃO, ERRO EM TEMPO DE EXECUÇÃO, ERRO EM TEMPO DE EXECUÇÃO - f1(1): ERRO EM TEMPO DE EXECUÇÃO, ERRO EM TEMPO DE EXECUÇÃO, ERRO EM TEMPO DE EXECUÇÃO



✓ O que é o ambiente de referência para uma sentença? *

1/1

- ☒ É o conjunto de todas as variáveis que são visíveis naquela sentença. ✓
- ☐ É o conjunto de todas as variáveis locais acessíveis naquela sentença.
- ☐ É o conjunto de todas as variáveis presentes no escopo mas não acessíveis naquela sentença.
- ☐ É o conjunto de todas as variáveis não-locais acessíveis naquela sentença.

✓ O tipo de escopo oferecido em uma linguagem (estático ou dinâmico) pode interferir no ambiente de referência? *

1/1

- ☐ Não. Tanto no escopo dinâmico como no estático, o acesso às variáveis é definido pela sequência de chamadas realizadas.
- ☒ Sim. No escopo dinâmico, o ambiente de referência depende da sequência de chamadas realizadas em tempo de execução. ✓
- ☐ Não. O escopo dinâmico ou estático afeta apenas a visibilidade das variáveis, deixando o ambiente de referência intacto em ambos os casos.
- ☐ Sim. No escopo dinâmico, o ambiente de referência depende do escopo definido pelo código a ser executado.

1 de 1 pontos

Considere o seguinte trecho de código (o mesmo de uma questão anterior):

```
1: void f1(int a) {  
2:   if (a > 0) {  
3:     f2()  
4:   }  
5:   else {  
6:     f3()  
7:   }  
8: }  
9:  
10: void f2() {  
11:   int a = 0  
12:   int b = 1  
13:   f4()  
14: }  
15:  
16: void f3() {  
17:   int b = 3
```

```
18: int c = 5
19: f4()
20: }
21:
22: void f4() {
23:   print(a)
24:   print(b)
25:   print(c)
26: }
```

✓ Qual é o ambiente de referência nas linhas (13) e (19) considerando a chamada `f1()` e escopo dinâmico? (considere que `x(){j, g}` representa as variáveis `j` e `g` declaradas em `x()`) *

☐ L.13: `f2(){a}, f3(){b}` --- L.19: `f2(){a}, f3(){b, c}`

☒ L.13: `f2(){a, b}` --- L.19: `f2(){a}, f3(){b, c}` ✓

☐ L.13: `f2(){a, b}` --- L.19: `f2(){a}, f3(){b}`

☐ L.13: `f2(){a, b}` --- L.19: `f2(){a, b}, f3(){c}`

1 de 1 pontos

Considere o seguinte trecho de código Perl

```
sub f1() {
  my $v1;
  ...
}
```



✓ Considere as palavras reservadas ``my`` e ``local`` de Perl, que afetam a regra de escopo aplicado a uma variável. Neste código ... *

1/1

☐ ``my $v1`` faz com que a variável `v1` oculte variável `v1` que possa ter sido declarada nas funções que chamaram `f1()`, tornando-a oculta para quaisquer chamadas realizadas nas sentenças subsequentes.

☐ ``my $v1`` faz com que a variável `v1` não oculte um variável `v1` que possa ter sido declarada nas funções que chamaram `f1()`.

☒ ``my $v1`` faz com que a variável `v1` não interfira na avaliação de escopo dinâmico nas chamadas das sentenças subsequentes, tornando ``v1`` de `f1()` invisível dentro de quaisquer chamadas adicionais. ✓

☐ ``my $v1`` faz com que a variável `v1` interfira na avaliação de escopo dinâmico nas chamadas das sentenças subsequentes, tornando ``v1`` de `f1()` visível dentro de quaisquer chamadas adicionais. Se fosse utilizado ``local $v1`` o efeito seria o inverso.

1 de 1 pontos

Considere o seguinte trecho de código Perl:

```
$v1 = 7;

sub f1my() {
  my $v1 = 11;
  f2();
}

sub f1local() {
  local $v1 = 3;
  f2();
}

sub f2() {
  print $v1;
}

f1my();
f1local();
```



✓ Quais valores serão impressos na saída do programa? *

1/1

☐ 7 e 7

☒ 7 e 3



☐ 11 e 3

☐ 11 e 7

2 de 2 pontos

Considere o código Perl abaixo, que é o mesmo exibido na video-aula, exceto pela condição de `count`? A execução do código é o resultado da invocação de sub1() na última linha. Quando uma variável não existe, o `print` não exibe nada.

```
$count = 0;
```

```
$w = 33;
```

```
$v = 44;
```

```
$x = 0;
```

```
sub sub1 {
```

```
  local $x = 1;
```

```
  local $y = 2;
```

```
  print "v = $v\n";
```

```
  print "x = $x\n";
```

```
  print "y = $y\n";
```

```
  print "w = $w\n";
```

```
  print "k = $k\n";
```

```
  print "-----\n";
```

```
  $count = $count + 1;
```

```
  if ($count < 2) {
```

```
    sub2();
```

```
  }
```

```
}
```

```
sub sub2 {
```

```
  local $y = 111;
```

```
  local $w = 222;
```

```
  local $k = 333;
```

```
  sub1();
```

```
}
```

```
sub1();
```



✓ Quais serão os valores exibidos na PRIMEIRA invocação de sub1() para as 1/1 variáveis v, x, y, w e k, respectivamente. *

☐ 44, 1, 111, 33, nada

☒ 44, 1, 2, 33, nada ✓

☐ 44, 0, 2, 33, 333

☐ 44, 1, 2, 222, 333

☐ 44, 1, 2, 33, 333

☐ 44, 0, 2, 33, nada

✓ Para o mesmo código anterior, quais serão os valores exibidos na 1/1 SEGUNDA invocação de sub1(), feita dentro de sub2(), para as variáveis v, x, y, w e k, respectivamente. *

☒ 44, 1, 2, 222, 333 ✓

☐ 44, 0, 2, 33, nada

☐ 44, 1, 111, 33, nada

☐ 44, 0, 2, 33, 333

☐ 44, 1, 2, 33, 333

☐ 44, 1, 2, 33, nada

Este formulário foi criado em Universidade Federal de Goiás.

Google Formulários

